Submission deadline and procedure: refer to Blackboard.

Introduction

This is the second (of two) assessed exercise for 6CM504. For each of these modules, this exercise is worth a total of 50% of the module. You are required to attempt both of task 1 and task 2 below. Task 1 is worth 30% of your overall mark for the module. Task 2 is worth 20% of your overall mark for the module.

Submission – all students

Your submission for this exercise should consist of one or two source CSP scripts (task1_username.csp and task2_username.csp, where appropriate) and any accompanying documentation (such as the short report). Submission should be via the "Assessment 2" portal on Blackboard. The deadline for submission is given on Blackboard. You will have approximately two weeks to complete this assignment (including two lab sessions) – it is not expected that the lab sessions alone will allow enough time to develop your solution.

This is an individual exercise. The University of Derby operates strict rules regarding plagiarism and these will be enforced where necessary. You may not use generative AI in constructing your solutions – indeed it will not be helpful to do so.

Background

In the lectures, we introduced several industrial case studies – modelling protocols, modelling component failure, and modelling source code. We demonstrated how CSP could be used to build models of various systems, and FDR used to analyse properties of interest. In this lab exercise you will develop these models further and explore how we improve our analysis – both to confirm properties of robustness, and to expose problematic scenarios.

Task 1

In lectures we presented Dekker's algorithm and some code that implemented Dekker's algorithm. This algorithm implements a mutual exclusion protocol for two processes, using only shared memory.

In the first part of this exercise, you should ensure you have a working model of Dekker's algorithm in FDR, and that you can verify that your model is correct using an appropriate safety property modelled as traces refinement assertion.

Modern compilers are very clever pieces of software – they can optimise code and introduce many efficiencies that are very difficult to implement, or to spot, at source code level. For instance, it can be possible to detect that a variable is not accessed within a particular scope and for that variable to be automatically removed to give a space efficiency. Normally, this produces an entirely equivalent piece of code with the exception of the improved space efficiency – but this is not always the case in a shared memory environment.

This is a particular problem for Dekker's algorithm. In some environments it is – surprisingly – perfectly legal for a compiler to detect that the variables `flag1` and `flag2` are never accessed in

the loop and therefore remove the writes to those variables in each loop. Similarly, it is also legal for a compiler to detect that the variable `turn` is not accessed in the loop and perform a similar transformation. Clearly, the issue here is that the compiler has made some assumptions (erroneously as it turns out) and performed a transformation on the code that firstly means it can break, and secondly means the compiled version is not faithful to the original source code.

In a case such as this it is reasonable to investigate the optimisations a compiler may make and understand if they have any effect – particularly an erroneous effect – on the final system. For this task you are required to adapt your original model of Dekker's algorithm to reflect the possible compiler optimisations discussed above, and prove that the algorithm will break when these optimisations are turned on. Your model should consider the possibility that one, or both, or these optimisations may, or may not, have been turned on.

Task 2

Another limitation of Dekker's algorithm is that it is known to work only for two processes. When a mutual exclusion algorithm for more than two processes is needed, Peterson's algorithm may be used. This algorithm can be generalised to implement mutual exclusion over critical sections for an arbitrary n processes. The algorithm is as follows:

The algorithm uses two variables, flag, and turn. If flag[n] is true, this is taken to indicate that process n wishes to enter the critical section. Entry to the critical section is granted for process 0 if process 1 does not want to enter the critical section, or if it has given priority to process 0 by setting the variable turn to 0.

```
VAR level[n] : Int
VAR last_to_enter[n-1] : Int

Pi_gate =
     FOR l = 0 TO n-1
     BEGIN
          level[i] := l;
          last_to_enter[l] := i;
          WHILE last_to_enter[l] = I
                && there exists k such that
                    k != I and level[k] >= l DO nothing

     END
     <critical section>
     level[i] := -1;
```

Each process `n` has an integer variable `level[n]` (which should be implemented as a single writer/multiple reader atomic register, and `n-1` additional variables in similar registers. The level variables take on values up to `n-1`, where each value represents a position in a notional queue to enter the critical section. Processes advance up the queue one position at a time and when they are in position `n-1` may enter the critical section. To release the lock when in the critical section, process `i` sets `level[i]` to `-1`.

Submission deadline and procedure: refer to Blackboard.

Intuitively, it can be seen that the algorithm achieves mutual exclusion as process `i` exits the inner loop when there is either no process with a higher level than `level[i]` (meaning that the queue ahead is free) or when `i != last_to_enter[l]` so another process entered its queue level. At level zero even if all `n` processes entered the queue in position zero at the same time, no more than `n-1` would proceed to the next queue position. This continues until at the final position in the queue only one process is allowed to leave the queue and enter the critical section.

The algorithm satisfies the three properties of primary interest to us: mutual exclusion of the critical sections, no blocking of progress, and bounded waiting (that is, the wait does not result in a divergence).

In this task, you are required to build a system of three processes employing Peterson's algorithm to implement mutual exclusion of a critical section. You should prove that your algorithm correctly implements mutual exclusion, and that progress of processes in the system is not blocked by the algorithm (assuming the critical sections themselves do not block).